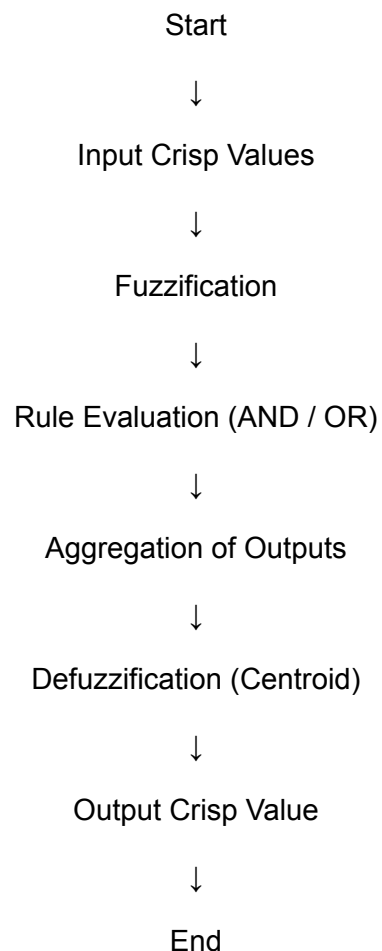


Introduction

In many real-life decision-making problems, inputs and outputs cannot be described using strict numerical boundaries. Concepts such as “good service”, “dirty clothes”, or “long wash time” are inherently vague and subjective. Fuzzy logic provides a way to model such uncertainty by allowing partial membership in linguistic categories and reasoning with approximate rules.

In this work, two fuzzy inference systems were implemented using pure Python without relying on any fuzzy logic library. The first system models a restaurant tipping scenario based on service quality and food quality. The second system controls the wash time of a washing machine based on the dirtiness of clothes and the load size. Both systems use Mamdani-type fuzzy inference and centroid defuzzification.

Fuzzy Inference System – Flowchart



1. Restaurant Tipping System:

1.1 Problem Description

The goal of this system is to determine a suitable tip percentage in a restaurant based on two subjective inputs:

- **Service quality** (poor, good, excellent)
- **Food quality** (bad, delicious)

The output variable is:

- **Tip percentage** (cheap, average, generous)

Since tipping depends on human perception and judgment, fuzzy logic is well-suited for capturing the gradual transitions between different levels of service and food quality.

1.2 Fuzzy Variables and Membership Functions

The input universes were defined in the range 0–10 for both service and food quality, while the output universe for tip percentage was defined in the range 5–25.

Triangular and trapezoidal membership functions were used to represent the linguistic terms. For example, service quality gradually transitions from “poor” to “good” and then to “excellent” instead of changing abruptly at fixed thresholds. Similarly, the tip percentage categories overlap to ensure smooth changes in the output.

```
def trimf(x, a, b, c):
    y = np.zeros_like(x, dtype=float)
    y[(x > a) & (x <= b)] = (x[(x > a) & (x <= b)] - a) / (b - a)
    y[(x > b) & (x < c)] = (c - x[(x > b) & (x < c)]) / (c - b)
    y[x == b] = 1
    return y

def trapmf(x, a, b, c, d):
    y = np.zeros_like(x, dtype=float)
    if b != a:
        y[(x >= a) & (x < b)] = (x[(x >= a) & (x < b)] - a) / (b - a)
    else:
        y[x == a] = 1
    y[(x >= b) & (x <= c)] = 1
    if d != c:
        y[(x > c) & (x <= d)] = (d - x[(x > c) & (x <= d)]) / (d - c)
    return y
```

```
service = Antecedent(np.arange(0, 11, 1), 'service')
food = Antecedent(np.arange(0, 11, 1), 'food')
tip = Consequent(np.arange(5, 26, 1), 'tip')

service['poor'] = trapmf(service.universe, 0, 0, 2, 4)
service['good'] = trimf(service.universe, 2, 5, 8)
service['excellent'] = trapmf(service.universe, 6, 8, 10, 10)

food['bad'] = trapmf(food.universe, 0, 0, 3, 7)
food['delicious'] = trapmf(food.universe, 3, 7, 10, 10)

tip['cheap'] = trapmf(tip.universe, 5, 5, 10, 13)
tip['average'] = trimf(tip.universe, 10, 15, 20)
tip['generous'] = trapmf(tip.universe, 17, 20, 25, 25)
```

1.3 Rule Base

A small set of intuitive fuzzy rules was constructed to model human tipping behavior. Examples include:

1. If service is poor or food is bad, then the tip should be cheap.

2. If service is good, then the tip should be average.
3. If service is excellent or food is delicious, then the tip should be generous.

```
rule1 = Rule([(service, 'poor'), (food, 'bad')], (tip, 'cheap'), operator="OR")
rule2 = Rule([(service, 'good')], (tip, 'average'), operator="AND")
rule3 = Rule([(service, 'excellent'), (food, 'delicious')], (tip, 'generous'), operator="OR")
```

```
class Antecedent:
    def __init__(self, universe, name):
        self.universe = universe
        self.name = name
        self.mf = {}

    def __getitem__(self, label, mf_values):
        self.mf[label] = mf_values

class Consequent(Antecedent):
    pass

class Rule:
    def __init__(self, antecedents, consequent, operator="AND"):
        self.antecedents = antecedents
        self.consequent = consequent
        self.operator = operator

class ControlSystem:
    def __init__(self, rules):
        self.rules = rules

class ControlSystemSimulation:
    def __init__(self, system):
        self.system = system
        self.input = {}
        self.output = {}
```

1.4 Inference and Defuzzification

For each rule, the degree of activation was computed using minimum (for AND) and maximum (for OR) operators. The resulting fuzzy output sets were clipped according to the firing strength of each rule and then aggregated using the maximum operator.

Finally, the centroid method was applied to convert the aggregated fuzzy output into a single crisp tip percentage. This method produces smooth and stable outputs and avoids sudden jumps in the final decision.

```
# Fuzzification + Rule Evaluation + Aggregation + Defuzzification
def compute(self):
    aggregated = np.zeros_like(self.system.rules[0].consequent[0].universe)

    # Inference
    for rule in self.system.rules:
        degrees = []
        for var, label in rule.antecedents:
            degree = np.interp(self.input[var.name], var.universe, var.mf[label])
            degrees.append(degree)

        firing_strength = min(degrees) if rule.operator == "AND" else max(degrees)

        out_var, out_label = rule.consequent
        clipped = np.minimum(firing_strength, out_var.mf[out_label])
        aggregated = np.maximum(aggregated, clipped)

    # Defuzzification
    self.output[out_var.name] = np.sum(out_var.universe * aggregated) / np.sum(aggregated)
```

1.5 Sample Test Case

For the test input where service quality was set to 3 and food quality was set to 6, the fuzzy system generated a tip value of approximately 16.08%. This value lies between the “average” and “generous” tip ranges. The result matches common human judgment, as moderately good service combined with good food usually leads to a slightly higher-than-average tip. This example shows how the system smoothly combines multiple fuzzy influences to produce a realistic output.

```
tipping_ctrl = ControlSystem([rule1, rule2, rule3])
tipping_sim = ControlSystemSimulation(tipping_ctrl)

# Fuzzification (inputs)
tipping_sim.input['service'] = 3
tipping_sim.input['food'] = 6

tipping_sim.compute()

print("tip output", round(tipping_sim.output['tip'], 2))

tip output 16.08
```

2. Fuzzy Washing Machine Controller

2.1 Problem Description

The washing machine controller determines the appropriate wash time based on:

- **Dirtiness of clothes** (low, medium, high)
- **Load size** (small, medium, large)

The output variable is:

- **Wash time** (short, medium, long)

This problem also involves subjective and gradual transitions, making fuzzy logic a suitable approach.

2.2 Fuzzy Variables and Membership Functions

The dirtiness and load size were represented on a normalized scale from 0 to 10, while wash time was defined over a continuous range (e.g., 0 to 60 minutes).

Overlapping triangular and trapezoidal membership functions were used so that, for example, a load can be partially “medium” and partially “large” at the same time. This reflects the real-world situation where the boundary between categories is not sharp.

```
def trimf(x, a, b, c):
    y = np.zeros_like(x, dtype=float)
    y[(x > a) & (x <= b)] = (x[(x > a) & (x <= b)] - a) / (b - a)
    y[(x > b) & (x < c)] = (c - x[(x > b) & (x < c)]) / (c - b)
    y[x == b] = 1
    return y

def trapmf(x, a, b, c, d):
    y = np.zeros_like(x, dtype=float)
    if b != a:
        y[(x >= a) & (x < b)] = (x[(x >= a) & (x < b)] - a) / (b - a)
    else:
        y[x == a] = 1
    y[(x >= b) & (x <= c)] = 1
    if d != c:
        y[(x > c) & (x <= d)] = (d - x[(x > c) & (x <= d)]) / (d - c)
    return y
```

```
dirtyiness = Antecedent(np.arange(0, 11, 1), 'dirtyiness')
load_size = Antecedent(np.arange(0, 11, 1), 'load_size')
wash_time = Consequent(np.arange(0, 61, 1), 'wash_time')

dirtyiness['low'] = trapmf(dirtyiness.universe, 0, 0, 3, 5)
dirtyiness['medium'] = trimf(dirtyiness.universe, 3, 5, 7, 10)
dirtyiness['high'] = trapmf(dirtyiness.universe, 6, 8, 10, 10)

load_size['small'] = trapmf(load_size.universe, 0, 0, 3, 5)
load_size['medium'] = trimf(load_size.universe, 3, 5, 7, 10)
load_size['large'] = trapmf(load_size.universe, 6, 8, 10, 10)

wash_time['short'] = trapmf(wash_time.universe, 0, 0, 15, 25)
wash_time['medium'] = trimf(wash_time.universe, 20, 30, 40, 60)
wash_time['long'] = trapmf(wash_time.universe, 35, 45, 60, 60)
```

2.3 Rule Base

The fuzzy rule base for the washing machine controller was constructed based on intuitive washing requirements. The complete set of rules used in the controller is listed below:

1. Rule 1: If dirtyiness is low AND load size is small, then wash time is short.
2. Rule 2: If dirtyiness is low AND load size is medium, then wash time is medium.
3. Rule 3: If dirtyiness is medium AND load size is small, then wash time is medium.
4. Rule 4: If dirtyiness is medium AND load size is large, then wash time is long.
5. Rule 5: If dirtyiness is high AND load size is medium, then wash time is long.
6. Rule 6: If dirtyiness is high AND load size is large, then wash time is long.
7. Rule 7: If dirtyiness is high OR load size is large, then wash time is long.

```
rule1 = Rule([(dirtyiness, 'low'), (load_size, 'small')], (wash_time, 'short'), operator="AND")
rule2 = Rule([(dirtyiness, 'low'), (load_size, 'medium')], (wash_time, 'medium'), operator="AND")
rule3 = Rule([(dirtyiness, 'medium'), (load_size, 'small')], (wash_time, 'medium'), operator="AND")
rule4 = Rule([(dirtyiness, 'medium'), (load_size, 'large')], (wash_time, 'long'), operator="AND")
rule5 = Rule([(dirtyiness, 'high'), (load_size, 'medium')], (wash_time, 'long'), operator="AND")
rule6 = Rule([(dirtyiness, 'high'), (load_size, 'large')], (wash_time, 'long'), operator="AND")
rule7 = Rule([(dirtyiness, 'high'), (load_size, 'large')], (wash_time, 'long'), operator="OR")
```

```

class Antecedent:
    def __init__(self, universe, name):
        self.universe = universe
        self.name = name
        self.mf = {}

    def __setitem__(self, label, mf_values):
        self.mf[label] = mf_values

class Consequent(Antecedent):
    pass

class Rule:
    def __init__(self, antecedents, consequent, operator="AND"):
        self.antecedents = antecedents
        self.consequent = consequent
        self.operator = operator

class ControlSystem:
    def __init__(self, rules):
        self.rules = rules

class ControlSystemSimulation:
    def __init__(self, system):
        self.system = system
        self.input = {}
        self.output = {}

```

2.4 Inference and Defuzzification

Similar to the tipping system, Mamdani inference was applied. The activation of each rule was computed using minimum and maximum operators. The fuzzy output sets were clipped and aggregated to form a combined output distribution.

Centroid defuzzification was then used to obtain a crisp wash time. This ensures smooth control behavior and avoids abrupt changes in washing duration when input values change slightly.

```

def compute(self):
    aggregated = np.zeros_like(self.system.rules[0].consequent[0].universe)

    for rule in self.system.rules:
        degrees = []
        for var, label in rule.antecedents:
            degree = np.interp(self.input[var.name], var.universe, var.mf[label])
            degrees.append(degree)

        firing_strength = min(degrees) if rule.operator == "AND" else max(degrees)

        out_var, out_label = rule.consequent
        clipped = np.minimum(firing_strength, out_var.mf[out_label])
        aggregated = np.maximum(aggregated, clipped)

    self.output[out_var.name] = np.sum(out_var.universe * aggregated) / np.sum(aggregated)

```

2.5 Sample Test Case

- (2, 2): Short wash time
- (3, 6): Medium wash time
- (5, 4): Medium wash time
- (7, 6): Long wash time
- (9, 9): Long wash time

The fuzzy controller adjusts wash time smoothly based on dirtiness and load size using Mamdani inference. Wash duration increases logically for dirtier clothes or larger loads, reflecting practical washing behavior.

```
wm_ctrl = ControlSystem([rule1, rule2, rule3, rule4, rule5, rule6, rule7])
wm_sim = ControlSystemSimulation(wm_ctrl)

test_cases = [
    (2, 2),
    (3, 6),
    (5, 4),
    (7, 6),
    (9, 9)
]

for i, (d, l) in enumerate(test_cases, 1):
    wm_sim.input['dirtiness'] = d
    wm_sim.input['load_size'] = l
    wm_sim.compute()
    print(f"Test Case {i}: Dirtiness={d}, Load Size={l} -> Wash Time = {round(wm_sim.output['wash_time'], 2)}")

Test Case 1: Dirtiness=2, Load Size=2 -> Wash Time = 9.95
Test Case 2: Dirtiness=3, Load Size=6 -> Wash Time = 30.0
Test Case 3: Dirtiness=5, Load Size=4 -> Wash Time = 30.0
Test Case 4: Dirtiness=7, Load Size=6 -> Wash Time = 48.96
Test Case 5: Dirtiness=9, Load Size=9 -> Wash Time = 50.05
```

Discussion

Both systems demonstrate how fuzzy logic can handle imprecise and subjective inputs more naturally than traditional rule-based systems with hard thresholds. The overlapping membership functions allow smooth transitions between categories, and the rule-based structure makes the systems easy to interpret and extend.

Using centroid defuzzification provides stable numeric outputs, which is particularly useful for control applications such as wash time selection. The manual implementation without external libraries also helps in understanding the internal working of fuzzy inference systems in detail.

Conclusion

This study implemented two fuzzy inference systems using pure Python to solve real-life decision problems: restaurant tipping and washing machine control. The results show that fuzzy logic effectively models human-like reasoning and provides smooth, interpretable outputs. The approach can be extended to more complex systems by adding additional inputs, refining membership functions, or expanding the rule base.

Colab code link : [🔗 Fuzzy_ResturantTip.ipynb](#)

Colab code link : [🔗 Fuzzy_washingMachine.ipynb](#)